Phase 2 Models

Note

> Encoutered issues with Torch on arm64 processor, needed to run final cells on work computer-hence the filepath change

Data Load

In [6]:
```
ls "C:\Users\c45526\OneDrive - Textron\College\Grad School\Fall 2025\drive-download
```

```
 Volume in drive C is OSDisk
 Volume Serial Number is BED0-AE97

 Directory of C:\Users\c45526\OneDrive - Textron\College\Grad School\Fall 2025\drive
-download-20251209T140518Z-1-001\New folder\home-credit-default-risk

12/09/2025  09:25 AM    <DIR>          .
12/09/2025  08:41 AM    <DIR>          ..
12/09/2025  08:51 AM        26,567,651 application_test.csv
12/09/2025  08:54 AM       166,133,370 application_train.csv
12/09/2025  09:04 AM       170,016,717 bureau.csv
12/09/2025  09:12 AM       375,592,889 bureau_balance.csv
12/09/2025  09:21 AM       424,582,605 credit_card_balance.csv
12/09/2025  08:54 AM            37,383 HomeCredit_columns_description.csv
12/09/2025  09:25 AM       723,118,349 installments_payments.csv
12/09/2025  09:20 AM       392,703,158 POS_CASH_balance.csv
12/09/2025  09:21 AM       404,973,293 previous_application.csv
12/09/2025  08:55 AM           536,202 sample_submission.csv
              10 File(s)  2,684,261,617 bytes
               2 Dir(s)  480,607,346,688 bytes free
```

In [7]:
```
DATA_DIR=r"C:\Users\c45526\OneDrive - Textron\College\Grad School\Fall 2025\drive-d
```

In [8]:
```
#%pip install pandas numpy matplotlib seaborn scikit-learn
```

In [9]:
```
# Import data from zip file
from pandas import read_csv

# Removed 'chunksize' so these load as proper DataFrames
application_train = read_csv(r'C:\Users\c45526\OneDrive - Textron\College\Grad Scho
application_test = read_csv(r'C:\Users\c45526\OneDrive - Textron\College\Grad Schoo
bureau = read_csv(r"C:\Users\c45526\OneDrive - Textron\College\Grad School\Fall 202
bureau_balance = read_csv(r"C:\Users\c45526\OneDrive - Textron\College\Grad School\
credit_card_balance = read_csv(r"C:\Users\c45526\OneDrive - Textron\College\Grad Sc
installments_payments = read_csv(r"C:\Users\c45526\OneDrive - Textron\College\Grad
POS_CASH_balance = read_csv(r"C:\Users\c45526\OneDrive - Textron\College\Grad Schoo
previous_application = read_csv(r"C:\Users\c45526\OneDrive - Textron\College\Grad S
```

```
sample_submission = read_csv(r"C:\Users\c45526\OneDrive - Textron\College\Grad Scho

# submission = read_csv('submission.csv') # Muted until final submission
HomeCredit_columns_description = read_csv(r'C:\Users\c45526\OneDrive - Textron\Coll

print("✅ Data loaded successfully as DataFrames.")
print(f"application_train shape: {application_train.shape}")
```

```
✅ Data loaded successfully as DataFrames.
application_train shape: (307511, 122)
```

In [10]:
```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import os
import zipfile
from sklearn.base import BaseEstimator, TransformerMixin
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler
from sklearn.pipeline import Pipeline, FeatureUnion
from pandas.plotting import scatter_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
import warnings
warnings.filterwarnings('ignore')

def load_data(in_path, name):
    df = pd.read_csv(in_path)
    print(f"{name}: shape is {df.shape}")
    print(df.info())
    display(df.head(5))
    return df
```

In [11]:
```
datasets={}  # lets store the datasets in a dictionary so we can keep track of them
ds_name = 'application_train'
DATA_DIR=f"{DATA_DIR}/home-credit-default-risk/"
datasets[ds_name] = load_data(os.path.join(DATA_DIR, f'{ds_name}.csv'), ds_name)
#datasets = pd.read_csv('application_train.csv')
#datasets['application_train'].shape
```

```
application_train: shape is (307511, 122)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 307511 entries, 0 to 307510
Columns: 122 entries, SK_ID_CURR to AMT_REQ_CREDIT_BUREAU_YEAR
dtypes: float64(65), int64(41), object(16)
memory usage: 286.2+ MB
None
```

	SK_ID_CURR	TARGET	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	FLAG_OV
0	100002	1	Cash loans	M	N	
1	100003	0	Cash loans	F	N	
2	100004	0	Revolving loans	M	Y	
3	100006	0	Cash loans	F	N	
4	100007	0	Cash loans	M	N	

5 rows × 122 columns

In [12]:
```
ds_names = ("application_train", "application_test", "bureau","bureau_balance","cre
            "previous_application","POS_CASH_balance")

for ds_name in ds_names:
    datasets[ds_name] = load_data(os.path.join(DATA_DIR, f'{ds_name}.csv'), ds_name
```

```
application_train: shape is (307511, 122)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 307511 entries, 0 to 307510
Columns: 122 entries, SK_ID_CURR to AMT_REQ_CREDIT_BUREAU_YEAR
dtypes: float64(65), int64(41), object(16)
memory usage: 286.2+ MB
None
```

	SK_ID_CURR	TARGET	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	FLAG_OV
0	100002	1	Cash loans	M	N	
1	100003	0	Cash loans	F	N	
2	100004	0	Revolving loans	M	Y	
3	100006	0	Cash loans	F	N	
4	100007	0	Cash loans	M	N	

5 rows × 122 columns

```
application_test: shape is (48744, 121)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48744 entries, 0 to 48743
Columns: 121 entries, SK_ID_CURR to AMT_REQ_CREDIT_BUREAU_YEAR
dtypes: float64(65), int64(40), object(16)
memory usage: 45.0+ MB
None
```

	SK_ID_CURR	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALT
0	100001	Cash loans	F	N	
1	100005	Cash loans	M	N	
2	100013	Cash loans	M	Y	
3	100028	Cash loans	F	N	
4	100038	Cash loans	M	Y	I

5 rows × 121 columns

```
bureau: shape is (1716428, 17)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1716428 entries, 0 to 1716427
Data columns (total 17 columns):
 #   Column                  Dtype  
---  ------                  -----  
 0   SK_ID_CURR              int64  
 1   SK_ID_BUREAU            int64  
 2   CREDIT_ACTIVE           object 
 3   CREDIT_CURRENCY         object 
 4   DAYS_CREDIT             int64  
 5   CREDIT_DAY_OVERDUE      int64  
 6   DAYS_CREDIT_ENDDATE     float64
 7   DAYS_ENDDATE_FACT       float64
 8   AMT_CREDIT_MAX_OVERDUE  float64
 9   CNT_CREDIT_PROLONG      int64  
 10  AMT_CREDIT_SUM          float64
 11  AMT_CREDIT_SUM_DEBT     float64
 12  AMT_CREDIT_SUM_LIMIT    float64
 13  AMT_CREDIT_SUM_OVERDUE  float64
 14  CREDIT_TYPE             object 
 15  DAYS_CREDIT_UPDATE      int64  
 16  AMT_ANNUITY             float64
dtypes: float64(8), int64(6), object(3)
memory usage: 222.6+ MB
None
```

	SK_ID_CURR	SK_ID_BUREAU	CREDIT_ACTIVE	CREDIT_CURRENCY	DAYS_CREDIT	CREDIT_D
0	215354	5714462	Closed	currency 1	-497	
1	215354	5714463	Active	currency 1	-208	
2	215354	5714464	Active	currency 1	-203	
3	215354	5714465	Active	currency 1	-203	
4	215354	5714466	Active	currency 1	-629	

```
bureau_balance: shape is (27299925, 3)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27299925 entries, 0 to 27299924
Data columns (total 3 columns):
 #   Column          Dtype
---  ------          -----
 0   SK_ID_BUREAU    int64
 1   MONTHS_BALANCE  int64
 2   STATUS          object
dtypes: int64(2), object(1)
memory usage: 624.8+ MB
None
```

	SK_ID_BUREAU	MONTHS_BALANCE	STATUS
0	5715448	0	C
1	5715448	-1	C
2	5715448	-2	C
3	5715448	-3	C
4	5715448	-4	C

```
credit_card_balance: shape is (3840312, 23)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3840312 entries, 0 to 3840311
Data columns (total 23 columns):
 #   Column                       Dtype
---  ------                       -----
 0   SK_ID_PREV                   int64
 1   SK_ID_CURR                   int64
 2   MONTHS_BALANCE               int64
 3   AMT_BALANCE                  float64
 4   AMT_CREDIT_LIMIT_ACTUAL      int64
 5   AMT_DRAWINGS_ATM_CURRENT     float64
 6   AMT_DRAWINGS_CURRENT         float64
 7   AMT_DRAWINGS_OTHER_CURRENT   float64
 8   AMT_DRAWINGS_POS_CURRENT     float64
 9   AMT_INST_MIN_REGULARITY      float64
 10  AMT_PAYMENT_CURRENT          float64
 11  AMT_PAYMENT_TOTAL_CURRENT    float64
 12  AMT_RECEIVABLE_PRINCIPAL     float64
 13  AMT_RECIVABLE                float64
 14  AMT_TOTAL_RECEIVABLE         float64
 15  CNT_DRAWINGS_ATM_CURRENT     float64
 16  CNT_DRAWINGS_CURRENT         int64
 17  CNT_DRAWINGS_OTHER_CURRENT   float64
 18  CNT_DRAWINGS_POS_CURRENT     float64
 19  CNT_INSTALMENT_MATURE_CUM    float64
 20  NAME_CONTRACT_STATUS         object
 21  SK_DPD                       int64
 22  SK_DPD_DEF                   int64
dtypes: float64(15), int64(7), object(1)
memory usage: 673.9+ MB
None
```

	SK_ID_PREV	SK_ID_CURR	MONTHS_BALANCE	AMT_BALANCE	AMT_CREDIT_LIMIT_ACTUA
0	2562384	378907	-6	56.970	13500
1	2582071	363914	-1	63975.555	4500
2	1740877	371185	-7	31815.225	45000
3	1389973	337855	-4	236572.110	22500
4	1891521	126868	-1	453919.455	45000

5 rows × 23 columns

```
installments_payments: shape is (13605401, 8)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13605401 entries, 0 to 13605400
Data columns (total 8 columns):
 #   Column                  Dtype
---  ------                  -----
 0   SK_ID_PREV              int64
 1   SK_ID_CURR              int64
 2   NUM_INSTALMENT_VERSION  float64
 3   NUM_INSTALMENT_NUMBER   int64
 4   DAYS_INSTALMENT         float64
 5   DAYS_ENTRY_PAYMENT      float64
 6   AMT_INSTALMENT          float64
 7   AMT_PAYMENT             float64
dtypes: float64(5), int64(3)
memory usage: 830.4 MB
None
```

	SK_ID_PREV	SK_ID_CURR	NUM_INSTALMENT_VERSION	NUM_INSTALMENT_NUMBER	DAY
0	1054186	161674	1.0	6	
1	1330831	151639	0.0	34	
2	2085231	193053	2.0	1	
3	2452527	199697	1.0	3	
4	2714724	167756	1.0	2	

```
previous_application: shape is (1670214, 37)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1670214 entries, 0 to 1670213
Data columns (total 37 columns):
 #   Column                       Non-Null Count    Dtype
---  ------                       --------------    -----
 0   SK_ID_PREV                   1670214 non-null  int64
 1   SK_ID_CURR                   1670214 non-null  int64
 2   NAME_CONTRACT_TYPE           1670214 non-null  object
 3   AMT_ANNUITY                  1297979 non-null  float64
 4   AMT_APPLICATION              1670214 non-null  float64
 5   AMT_CREDIT                   1670213 non-null  float64
 6   AMT_DOWN_PAYMENT             774370 non-null   float64
 7   AMT_GOODS_PRICE              1284699 non-null  float64
 8   WEEKDAY_APPR_PROCESS_START   1670214 non-null  object
 9   HOUR_APPR_PROCESS_START      1670214 non-null  int64
 10  FLAG_LAST_APPL_PER_CONTRACT  1670214 non-null  object
 11  NFLAG_LAST_APPL_IN_DAY       1670214 non-null  int64
 12  RATE_DOWN_PAYMENT            774370 non-null   float64
 13  RATE_INTEREST_PRIMARY        5951 non-null     float64
 14  RATE_INTEREST_PRIVILEGED     5951 non-null     float64
 15  NAME_CASH_LOAN_PURPOSE       1670214 non-null  object
 16  NAME_CONTRACT_STATUS         1670214 non-null  object
 17  DAYS_DECISION                1670214 non-null  int64
 18  NAME_PAYMENT_TYPE            1670214 non-null  object
 19  CODE_REJECT_REASON           1670214 non-null  object
 20  NAME_TYPE_SUITE              849809 non-null   object
 21  NAME_CLIENT_TYPE             1670214 non-null  object
 22  NAME_GOODS_CATEGORY          1670214 non-null  object
 23  NAME_PORTFOLIO               1670214 non-null  object
 24  NAME_PRODUCT_TYPE            1670214 non-null  object
 25  CHANNEL_TYPE                 1670214 non-null  object
 26  SELLERPLACE_AREA             1670214 non-null  int64
 27  NAME_SELLER_INDUSTRY         1670214 non-null  object
 28  CNT_PAYMENT                  1297984 non-null  float64
 29  NAME_YIELD_GROUP             1670214 non-null  object
 30  PRODUCT_COMBINATION          1669868 non-null  object
 31  DAYS_FIRST_DRAWING           997149 non-null   float64
 32  DAYS_FIRST_DUE               997149 non-null   float64
 33  DAYS_LAST_DUE_1ST_VERSION    997149 non-null   float64
 34  DAYS_LAST_DUE                997149 non-null   float64
 35  DAYS_TERMINATION             997149 non-null   float64
 36  NFLAG_INSURED_ON_APPROVAL    997149 non-null   float64
dtypes: float64(15), int64(6), object(16)
memory usage: 471.5+ MB
None
```

	SK_ID_PREV	SK_ID_CURR	NAME_CONTRACT_TYPE	AMT_ANNUITY	AMT_APPLICATION	A
0	2030495	271877	Consumer loans	1730.430	17145.0	
1	2802425	108129	Cash loans	25188.615	607500.0	
2	2523466	122040	Cash loans	15060.735	112500.0	
3	2819243	176158	Cash loans	47041.335	450000.0	
4	1784265	202054	Cash loans	31924.395	337500.0	

5 rows × 37 columns

```
POS_CASH_balance: shape is (10001358, 8)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10001358 entries, 0 to 10001357
Data columns (total 8 columns):
 #   Column                 Dtype
---  ------                 -----
 0   SK_ID_PREV             int64
 1   SK_ID_CURR             int64
 2   MONTHS_BALANCE         int64
 3   CNT_INSTALMENT         float64
 4   CNT_INSTALMENT_FUTURE  float64
 5   NAME_CONTRACT_STATUS   object
 6   SK_DPD                 int64
 7   SK_DPD_DEF             int64
dtypes: float64(2), int64(5), object(1)
memory usage: 610.4+ MB
None
```

	SK_ID_PREV	SK_ID_CURR	MONTHS_BALANCE	CNT_INSTALMENT	CNT_INSTALMENT_FUTU
0	1803195	182943	-31	48.0	4
1	1715348	367990	-33	36.0	3
2	1784872	397406	-32	12.0	
3	1903291	269225	-35	48.0	4
4	2341044	334279	-35	36.0	3

In [13]:
```
datasets.keys()
```

Out[13]:
```
dict_keys(['application_train', 'application_test', 'bureau', 'bureau_balance', 'credit_card_balance', 'installments_payments', 'previous_application', 'POS_CASH_balance'])
```

Previous Submission File

In [14]:
```
# Check if 'previous_application' exists in datasets, if not load it
if 'previous_application' not in datasets:
        ds_name = 'previous_application'
        datasets[ds_name] = load_data(os.path.join(DATA_DIR, f'{ds_name}.csv'), ds_

appsDF = datasets["previous_application"]
display(appsDF.head())
print(f"{appsDF.shape[0]:,} rows, {appsDF.shape[1]:,} columns")
```

	SK_ID_PREV	SK_ID_CURR	NAME_CONTRACT_TYPE	AMT_ANNUITY	AMT_APPLICATION	A
0	2030495	271877	Consumer loans	1730.430	17145.0	
1	2802425	108129	Cash loans	25188.615	607500.0	
2	2523466	122040	Cash loans	15060.735	112500.0	
3	2819243	176158	Cash loans	47041.335	450000.0	
4	1784265	202054	Cash loans	31924.395	337500.0	

5 rows × 37 columns

```
1,670,214 rows, 37 columns
```

Missing Values

In [15]:
```
# Create aggregate features (via pipeline)
class prevAppsFeaturesAggregater(BaseEstimator, TransformerMixin):
    def __init__(self, features=None): # no *args or **kargs
        self.features = features
        self.agg_op_features = {}
        for f in features:
            self.agg_op_features[f] = ["min", "max", "mean"]

    def fit(self, X, y=None):
        return self

    def transform(self, X, y=None):
        #from IPython.core.debugger import Pdb as pdb;    pdb().set_trace() #breakp
        result = X.groupby(["SK_ID_CURR"]).agg(self.agg_op_features)
        result.columns = ['_'.join(col).strip() for col in result.columns.values]
        result = result.reset_index()
        result['range_AMT_APPLICATION'] = result['AMT_APPLICATION_max'] - result['A
        return result # return dataframe with the join key "SK_ID_CURR"


from sklearn.pipeline import make_pipeline
def test_driver_prevAppsFeaturesAggregater(df, features):
    print(f"df.shape: {df.shape}\n")
    print(f"df[{features}][0:5]: \n{df[features][0:5]}")
    test_pipeline = make_pipeline(prevAppsFeaturesAggregater(features))
    return(test_pipeline.fit_transform(df))
```

```
features = ['AMT_ANNUITY', 'AMT_APPLICATION']
features = ['AMT_ANNUITY',
       'AMT_APPLICATION', 'AMT_CREDIT', 'AMT_DOWN_PAYMENT', 'AMT_GOODS_PRICE',
       'RATE_DOWN_PAYMENT', 'RATE_INTEREST_PRIMARY',
       'RATE_INTEREST_PRIVILEGED', 'DAYS_DECISION', 'NAME_PAYMENT_TYPE',
       'CNT_PAYMENT',
       'DAYS_FIRST_DRAWING', 'DAYS_FIRST_DUE', 'DAYS_LAST_DUE_1ST_VERSION',
       'DAYS_LAST_DUE', 'DAYS_TERMINATION']
features = ['AMT_ANNUITY', 'AMT_APPLICATION']
res = test_driver_prevAppsFeaturesAggregater(appsDF, features)
print(f"HELLO")
print(f"Test driver: \n{res[0:10]}")
print(f"input[features][0:10]: \n{appsDF[0:10]}")


# QUESTION, should we lower case df['OCCUPATION_TYPE'] as Sales staff != 'Sales Sta
```

```
df.shape: (1670214, 37)

df[['AMT_ANNUITY', 'AMT_APPLICATION']][0:5]:
   AMT_ANNUITY  AMT_APPLICATION
0     1730.430          17145.0
1    25188.615         607500.0
2    15060.735         112500.0
3    47041.335         450000.0
4    31924.395         337500.0
HELLO
Test driver:
   SK_ID_CURR  AMT_ANNUITY_min  AMT_ANNUITY_max  AMT_ANNUITY_mean  \
0      100001         3951.000         3951.000       3951.000000
1      100002         9251.775         9251.775       9251.775000
2      100003         6737.310        98356.995      56553.990000
3      100004         5357.250         5357.250       5357.250000
4      100005         4813.200         4813.200       4813.200000
5      100006         2482.920        39954.510      23651.175000
6      100007         1834.290        22678.785      12278.805000
7      100008         8019.090        25309.575      15839.696250
8      100009         7435.845        17341.605      10051.412143
9      100010        27463.410        27463.410      27463.410000

   AMT_APPLICATION_min  AMT_APPLICATION_max  AMT_APPLICATION_mean  \
0              24835.5              24835.5          24835.500000
1             179055.0             179055.0         179055.000000
2              68809.5             900000.0         435436.500000
3              24282.0              24282.0          24282.000000
4                  0.0              44617.5          22308.750000
5                  0.0             688500.0         272203.260000
6              17176.5             247500.0         150530.250000
7                  0.0             450000.0         155701.800000
8              40455.0             110160.0          76741.714286
9             247212.0             247212.0         247212.000000

   range_AMT_APPLICATION
0                    0.0
1                    0.0
2                 831190.5
3                    0.0
4                 44617.5
5                 688500.0
6                 230323.5
7                 450000.0
8                  69705.0
9                    0.0
input[features][0:10]:
   SK_ID_PREV  SK_ID_CURR NAME_CONTRACT_TYPE  AMT_ANNUITY  AMT_APPLICATION  \
0     2030495      271877     Consumer loans     1730.430          17145.0
1     2802425      108129         Cash loans    25188.615         607500.0
2     2523466      122040         Cash loans    15060.735         112500.0
3     2819243      176158         Cash loans    47041.335         450000.0
4     1784265      202054         Cash loans    31924.395         337500.0
5     1383531      199383         Cash loans    23703.930         315000.0
6     2315218      175704         Cash loans          NaN              0.0
7     1656711      296299         Cash loans          NaN              0.0
```

```
8     2367563      342292         Cash loans          NaN              0.0
9     2579447      334349         Cash loans          NaN              0.0

   AMT_CREDIT  AMT_DOWN_PAYMENT  AMT_GOODS_PRICE WEEKDAY_APPR_PROCESS_START  \
0     17145.0              0.0          17145.0                   SATURDAY
1    679671.0              NaN         607500.0                   THURSDAY
2    136444.5              NaN         112500.0                    TUESDAY
3    470790.0              NaN         450000.0                     MONDAY
4    404055.0              NaN         337500.0                   THURSDAY
5    340573.5              NaN         315000.0                   SATURDAY
6         0.0              NaN              NaN                    TUESDAY
7         0.0              NaN              NaN                     MONDAY
8         0.0              NaN              NaN                     MONDAY
9         0.0              NaN              NaN                   SATURDAY

   HOUR_APPR_PROCESS_START  ... NAME_SELLER_INDUSTRY  CNT_PAYMENT  \
0                       15  ...         Connectivity         12.0
1                       11  ...                  XNA         36.0
2                       11  ...                  XNA         12.0
3                        7  ...                  XNA         12.0
4                        9  ...                  XNA         24.0
5                        8  ...                  XNA         18.0
6                       11  ...                  XNA          NaN
7                        7  ...                  XNA          NaN
8                       15  ...                  XNA          NaN
9                       15  ...                  XNA          NaN

   NAME_YIELD_GROUP       PRODUCT_COMBINATION  DAYS_FIRST_DRAWING  \
0            middle  POS mobile with interest            365243.0
1       low_action          Cash X-Sell: low            365243.0
2              high         Cash X-Sell: high            365243.0
3            middle       Cash X-Sell: middle            365243.0
4              high         Cash Street: high                 NaN
5       low_normal          Cash X-Sell: low            365243.0
6               XNA                     Cash                 NaN
7               XNA                     Cash                 NaN
8               XNA                     Cash                 NaN
9               XNA                     Cash                 NaN

  DAYS_FIRST_DUE DAYS_LAST_DUE_1ST_VERSION  DAYS_LAST_DUE DAYS_TERMINATION  \
0          -42.0                    300.0          -42.0            -37.0
1         -134.0                    916.0       365243.0         365243.0
2         -271.0                     59.0       365243.0         365243.0
3         -482.0                   -152.0         -182.0           -177.0
4            NaN                      NaN            NaN              NaN
5         -654.0                   -144.0         -144.0           -137.0
6            NaN                      NaN            NaN              NaN
7            NaN                      NaN            NaN              NaN
8            NaN                      NaN            NaN              NaN
9            NaN                      NaN            NaN              NaN

  NFLAG_INSURED_ON_APPROVAL
0                       0.0
1                       1.0
2                       1.0
3                       1.0
```

```
4                       NaN
5                       1.0
6                       NaN
7                       NaN
8                       NaN
9                       NaN

[10 rows x 37 columns]
```

Pipeline Creation

In [16]:
```
# Lets make a pipeline to process previous applications and join to main datasets
prevApps_aggregater = prevAppsFeaturesAggregater(features=['AMT_ANNUITY', 'AMT_APPL
       'AMT_CREDIT', 'AMT_DOWN_PAYMENT', 'AMT_GOODS_PRICE',
       'RATE_DOWN_PAYMENT', 'RATE_INTEREST_PRIMARY', 'RATE_INTEREST_PRIVILEGED',
       'DAYS_DECISION', 'CNT_PAYMENT', 'DAYS_FIRST_DRAWING', 'DAYS_FIRST_DUE',
       'DAYS_LAST_DUE_1ST_VERSION', 'DAYS_LAST_DUE', 'DAYS_TERMINATION'])
```

In [17]:
```
# Next lets create a full pipeline to process the previous applications and join to
# Pipeline Creation
from sklearn.pipeline import make_pipeline
prevApps_pipeline = make_pipeline(prevApps_aggregater)
```

In [18]:
```
# Now we can process and join to main datasets
prevApps_aggregated = prevApps_pipeline.fit_transform(datasets["previous_applicatio
prevApps_aggregated.head()
```

Out[18]:

	SK_ID_CURR	AMT_ANNUITY_min	AMT_ANNUITY_max	AMT_ANNUITY_mean	AMT_APPL
0	100001	3951.000	3951.000	3951.000	
1	100002	9251.775	9251.775	9251.775	
2	100003	6737.310	98356.995	56553.990	
3	100004	5357.250	5357.250	5357.250	
4	100005	4813.200	4813.200	4813.200	

5 rows × 47 columns

In [19]:
```
# OHE on the joined dataset
prevApps_aggregated_ohe = pd.get_dummies(prevApps_aggregated, drop_first=True)
prevApps_aggregated_ohe.head()
```

Out[19]:

	SK_ID_CURR	AMT_ANNUITY_min	AMT_ANNUITY_max	AMT_ANNUITY_mean	AMT_APPL
0	100001	3951.000	3951.000	3951.000	
1	100002	9251.775	9251.775	9251.775	
2	100003	6737.310	98356.995	56553.990	
3	100004	5357.250	5357.250	5357.250	
4	100005	4813.200	4813.200	4813.200	

5 rows × 47 columns

Preprocesing Cont

In [20]:

```
# Split the provided training data into training and validation and test
# The kaggle evaluation test set has no labels
#
from sklearn.model_selection import train_test_split

use_application_data_ONLY = False #use joined data
selected_features = ['AMT_INCOME_TOTAL',  'AMT_CREDIT','DAYS_EMPLOYED','DAYS_BIRTH'
        'EXT_SOURCE_2','EXT_SOURCE_3','CODE_GENDER', 'FLAG_OWN_REALTY','FLAG_OWN_CA
                    'NAME_EDUCATION_TYPE','OCCUPATION_TYPE','NAME_INCOME_TYPE']

if use_application_data_ONLY:
    # just selected a few features for a baseline experiment
    X_train = datasets["application_train"][selected_features]
    y_train = datasets["application_train"]['TARGET']
    X_train, X_valid, y_train, y_valid = train_test_split(X_train, y_train, test_si
    X_train, X_test, y_train, y_test = train_test_split(X_train, y_train, test_size
    X_kaggle_test = datasets["application_test"][selected_features]
    # y_test = datasets["application_test"]['TARGET']   #why no  TARGET?!! (hint: k
else:
    # Use joined data
    X_train = datasets["application_train"][selected_features]
    y_train = datasets["application_train"]['TARGET']
    X_train, X_valid, y_train, y_valid = train_test_split(X_train, y_train, test_si
    X_train, X_test, y_train, y_test = train_test_split(X_train, y_train, test_size
    X_kaggle_test = datasets["application_test"][selected_features]
    # y_test = datasets["application_test"]['TARGET']   #why no  TARGET?!! (hint: k


print(f"X train           shape: {X_train.shape}")
print(f"X validation      shape: {X_valid.shape}")
print(f"X test            shape: {X_test.shape}")
print(f"X X_kaggle_test   shape: {X_kaggle_test.shape}")
```

```
X train           shape: (222176, 14)
X validation      shape: (46127, 14)
X test            shape: (39208, 14)
X X_kaggle_test   shape: (48744, 14)
```

In [21]:

```
from sklearn.base import BaseEstimator, TransformerMixin
import re

# Creates the following date features
# But could do so much more with these features
#    E.g.,
#      extract the domain address of the homepage and OneHotEncode it
#
# ['release_month','release_day','release_year', 'release_dayofweek','release_quart
class prep_OCCUPATION_TYPE(BaseEstimator, TransformerMixin):
    def __init__(self, features="OCCUPATION_TYPE"): # no *args or **kargs
        self.features = features
    def fit(self, X, y=None):
        return self  # nothing else to do
    def transform(self, X):
        df = pd.DataFrame(X, columns=self.features)
        #from IPython.core.debugger import Pdb as pdb;    pdb().set_trace() #breakp
        df['OCCUPATION_TYPE'] = df['OCCUPATION_TYPE'].apply(lambda x: 1. if x in ['
        #df.drop(self.features, axis=1, inplace=True)
        return np.array(df.values)  #return a Numpy Array to observe the pipeline p


from sklearn.pipeline import make_pipeline
features = ["OCCUPATION_TYPE"]
def test_driver_prep_OCCUPATION_TYPE():
    print(f"X_train.shape: {X_train.shape}\n")
    print(f"X_train['name'][0:5]: \n{X_train[features][0:5]}")
    test_pipeline = make_pipeline(prep_OCCUPATION_TYPE(features))
    return(test_pipeline.fit_transform(X_train))

x = test_driver_prep_OCCUPATION_TYPE()
print(f"Test driver: \n{test_driver_prep_OCCUPATION_TYPE()[0:10, :]}")
print(f"X_train['name'][0:10]: \n{X_train[features][0:10]}")


# QUESTION, should we lower case df['OCCUPATION_TYPE'] as Sales staff != 'Sales Sta
```

```
X_train.shape: (222176, 14)

X_train['name'][0:5]:
       OCCUPATION_TYPE
21614      Sales staff
209797        Laborers
17976              NaN
282543  Security staff
52206              NaN
X_train.shape: (222176, 14)

X_train['name'][0:5]:
       OCCUPATION_TYPE
21614      Sales staff
209797        Laborers
17976              NaN
282543  Security staff
52206              NaN
Test driver:
[[0.]
 [0.]
 [0.]
 [0.]
 [0.]
 [1.]
 [0.]
 [0.]
 [0.]
 [0.]]
X_train['name'][0:10]:
       OCCUPATION_TYPE
21614      Sales staff
209797        Laborers
17976              NaN
282543  Security staff
52206              NaN
152195        Managers
70364       Core staff
11643              NaN
45591       Core staff
93535         Laborers
```

In [22]:

```python
# Create a class to select numerical or categorical columns
# since Scikit-Learn doesn't handle DataFrames yet
class DataFrameSelector(BaseEstimator, TransformerMixin):
    def __init__(self, attribute_names):
        self.attribute_names = attribute_names
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        return X[self.attribute_names].values
# Identify the numeric features we wish to consider.
num_attribs = [
    'AMT_INCOME_TOTAL',  'AMT_CREDIT','DAYS_EMPLOYED','DAYS_BIRTH','EXT_SOURCE_1',
    'EXT_SOURCE_2','EXT_SOURCE_3']
```

```
num_pipeline = Pipeline([
        ('selector', DataFrameSelector(num_attribs)),
        ('imputer', SimpleImputer(strategy='mean')),
        ('std_scaler', StandardScaler()),
    ])
# Identify the categorical features we wish to consider.
cat_attribs = ['CODE_GENDER', 'FLAG_OWN_REALTY','FLAG_OWN_CAR','NAME_CONTRACT_TYPE'
               'NAME_EDUCATION_TYPE','OCCUPATION_TYPE','NAME_INCOME_TYPE']

# Notice handle_unknown="ignore" in OHE which ignore values from the validation/tes
# do NOT occur in the training set
cat_pipeline = Pipeline([
        ('selector', DataFrameSelector(cat_attribs)),
        #('imputer', SimpleImputer(strategy='most_frequent')),
        ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
        ('ohe', OneHotEncoder(sparse_output=False, handle_unknown="ignore"))
    ])

data_prep_pipeline = FeatureUnion(transformer_list=[
        ("num_pipeline", num_pipeline),
        ("cat_pipeline", cat_pipeline),
    ])
```

Models/Pipelines

Logistic Regression

In [23]:
```
%%time
# Logistic Regression Model Pipeline

np.random.seed(42)
full_pipeline_with_predictor = Pipeline([
        ("preparation", data_prep_pipeline),
        ("logistic", LogisticRegression(penalty='l2', class_weight='balanced', rand
        )
    ])
model_logistic = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
CPU times: total: 2.12 s
Wall time: 3.4 s
```

Random Forrest

In [24]:
```
%%time
# Random Forest Model Pipeline

from sklearn.ensemble import RandomForestClassifier
np.random.seed(42)
full_pipeline_with_predictor = Pipeline([
        ("preparation", data_prep_pipeline),
        ("random_forest", RandomForestClassifier(class_weight='balanced', random_st
```

```
    ])
model_rf = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
CPU times: total: 31.7 s
Wall time: 53.3 s
```

XGBoost

In [25]:
```
%%time
 # XGBoost Model Pipeline
#%pip install xgboost
from xgboost import XGBClassifier


np.random.seed(42)
full_pipeline_with_predictor = Pipeline([
        ("preparation", data_prep_pipeline),
        ("xgb", XGBClassifier(random_state=42,
                              use_label_encoder=False,
                              eval_metric='logloss',scale_pos_weight=11.47))
    ])
model_xgb = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
CPU times: total: 7.41 s
Wall time: 1.31 s
```

LightGBM

In [26]:
```
%%time
# LightGBM Model Pipeline
#%pip install lightgbm
from lightgbm import LGBMClassifier



np.random.seed(42)
full_pipeline_with_predictor = Pipeline([
        ("preparation", data_prep_pipeline),
        ("lgbm", LGBMClassifier(scale_pos_weight=11.47,random_state=42)) # Scale we
    ])
model_lgbm = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
[LightGBM] [Info] Number of positive: 17815, number of negative: 204361
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing wa
s 0.008353 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1801
[LightGBM] [Info] Number of data points in the train set: 222176, number of used fea
tures: 43
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.080184 -> initscore=-2.439847
[LightGBM] [Info] Start training from score -2.439847
CPU times: total: 3.58 s
Wall time: 1.18 s
```

Catboost

In [27]:
```
%%time
# CatBoost Model Pipeline
#%pip install catboost
from catboost import CatBoostClassifier
import catboost
np.random.seed(42)
full_pipeline_with_predictor = Pipeline([
        ("preparation", data_prep_pipeline),
        ("catboost", catboost.CatBoostClassifier(verbose=0,
        auto_class_weights='Balanced'))
    ])
model_cat = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
CPU times: total: 2min 21s
Wall time: 20.8 s
```

Model Comparison

In [28]:
```
from sklearn.metrics import roc_auc_score, precision_score, recall_score
import pandas as pd
import numpy as np

# ----------------- 1. Define Models and Storage -----------------
# Ensure all these models are trained before running this cell
models = {
    "Logistic Regression": model_logistic,
    "Random Forest": model_rf,
    "XGBoost": model_xgb,
    "LightGBM": model_lgbm,
    "CatBoost": model_cat  # Added CatBoost
}

results = []
threshold = 0.5 # Default threshold for classification

# ----------------- 2. Calculate Metrics for Each Model -----------------
# NOTE: Changed X_valid to X_test to match your earlier train_test_split code
print("Starting model evaluation on X_test...")

for name, model in models.items():
    try:
        # A. Get predicted probabilities (required for ROC AUC)
        # The [:, 1] gets the probability of the positive class (Target=1 / Default
        y_pred_proba = model.predict_proba(X_test)[:, 1]

        # B. Get hard predictions (required for Precision and Recall)
        y_pred = (y_pred_proba >= threshold).astype(int)

        # C. Calculate Metrics
        roc_auc = roc_auc_score(y_test, y_pred_proba)
```

```
        precision = precision_score(y_test, y_pred, zero_division=0)
        recall = recall_score(y_test, y_pred, zero_division=0)

        # D. Store Results
        results.append({
            'Model': name,
            'ROC_AUC': roc_auc,
            'Precision (T=0.5)': precision,
            'Recall (T=0.5)': recall
        })

        print(f"-> {name} evaluated.")
    except Exception as e:
        print(f"Error evaluating {name}: {e}")

# ----------------- 3. Display Results -----------------
results_df = pd.DataFrame(results)

# Sort by ROC_AUC to see the best model immediately
results_df_sorted = results_df.sort_values(by='ROC_AUC', ascending=False)

print("\n=======================================================")
print("           MODEL PERFORMANCE COMPARISON")
print("=======================================================")
display(results_df_sorted)
print("=======================================================")
```

```
Starting model evaluation on X_test...
-> Logistic Regression evaluated.
-> Random Forest evaluated.
-> XGBoost evaluated.
-> LightGBM evaluated.
-> CatBoost evaluated.

=======================================================
           MODEL PERFORMANCE COMPARISON
=======================================================
```

	Model	ROC_AUC	Precision (T=0.5)	Recall (T=0.5)
3	LightGBM	0.747997	0.169209	0.680534
4	CatBoost	0.741395	0.175812	0.629517
2	XGBoost	0.736451	0.169657	0.623140
0	Logistic Regression	0.736113	0.161223	0.661099
1	Random Forest	0.716173	0.510204	0.007592

```
=======================================================
```

Phase 3

RFM Feature ENG

In [29]:
```
def create_rfm_features(bureau_df, app_df):
    """Create and merge RFM features into application data."""
    # Frequency: Number of past loans
    frequency = bureau_df.groupby('SK_ID_CURR').size().reset_index(name='RFM_Freque

    # Monetary: Total past credit
    monetary = bureau_df.groupby('SK_ID_CURR')['AMT_CREDIT_SUM'].sum().reset_index(

    # Recency: Most recent loan (max DAYS_CREDIT)
    recency = bureau_df.groupby('SK_ID_CURR')['DAYS_CREDIT'].max().reset_index(name

    # Merge
    app_df = app_df.merge(frequency, on='SK_ID_CURR', how='left')
    app_df = app_df.merge(monetary, on='SK_ID_CURR', how='left')
    app_df = app_df.merge(recency, on='SK_ID_CURR', how='left')

    # Impute missing values
    app_df['RFM_Frequency'].fillna(0, inplace=True)
    app_df['RFM_Monetary'].fillna(0, inplace=True)
    app_df['RFM_Recency'].fillna(app_df['RFM_Recency'].min(), inplace=True)

    return app_df
```

In [30]:
```
create_rfm_features(datasets['bureau'],datasets['application_test'])
```

Out[30]:

	SK_ID_CURR	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_
0	100001	Cash loans	F	N	
1	100005	Cash loans	M	N	
2	100013	Cash loans	M	Y	
3	100028	Cash loans	F	N	
4	100038	Cash loans	M	Y	
...	...	...	...	...	
48739	456221	Cash loans	F	N	
48740	456222	Cash loans	F	N	
48741	456223	Cash loans	F	Y	
48742	456224	Cash loans	M	N	
48743	456250	Cash loans	F	Y	

48744 rows × 124 columns

Pipeline Update

In [31]:
```
# Create a class to select numerical or categorical columns
# since Scikit-Learn doesn't handle DataFrames yet
class DataFrameSelector(BaseEstimator, TransformerMixin):
    def __init__(self, attribute_names):
        self.attribute_names = attribute_names
    def fit(self, X, y=None):
        return self
    def transform(self, X):
        return X[self.attribute_names].values

# Identify the numeric features we wish to consider.
# UPDATE: Added RFM features here so the model actually uses them!
num_attribs = [
    'AMT_INCOME_TOTAL',  'AMT_CREDIT','DAYS_EMPLOYED','DAYS_BIRTH','EXT_SOURCE_1',
    'EXT_SOURCE_2','EXT_SOURCE_3',
    'RFM_Frequency', 'RFM_Monetary', 'RFM_Recency'
] ## Added new features

num_pipeline = Pipeline([
        ('selector', DataFrameSelector(num_attribs)),
        ('imputer', SimpleImputer(strategy='mean')),
        ('std_scaler', StandardScaler()),
    ])

# Identify the categorical features we wish to consider.
cat_attribs = ['CODE_GENDER', 'FLAG_OWN_REALTY','FLAG_OWN_CAR','NAME_CONTRACT_TYPE'
               'NAME_EDUCATION_TYPE','OCCUPATION_TYPE','NAME_INCOME_TYPE']

# Notice handle_unknown="ignore" in OHE which ignore values from the validation/tes
# do NOT occur in the training set
cat_pipeline = Pipeline([
        ('selector', DataFrameSelector(cat_attribs)),
        #('imputer', SimpleImputer(strategy='most_frequent')),
        ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
        ('ohe', OneHotEncoder(sparse_output=False, handle_unknown="ignore"))
    ])

data_prep_pipeline = FeatureUnion(transformer_list=[
        ("num_pipeline", num_pipeline),
        ("cat_pipeline", cat_pipeline),
    ])
```

In [32]:
```
# 1. Apply RFM features to the main training dataframe
# Note: creating a copy to avoid SettingWithCopy warnings
datasets['application_train'] = create_rfm_features(datasets['bureau'], datasets['a

# 2. Add the new feature names to your selection list
# (This list was originally defined way back in Cell 21)
rfm_features = ['RFM_Frequency', 'RFM_Monetary', 'RFM_Recency']
# Ensure we don't duplicate if you run this cell multiple times
for f in rfm_features:
    if f not in selected_features:
        selected_features.append(f)

# 3. Re-run Train/Test Split with the updated dataframe and feature list
```

```
X_train, X_valid, y_train, y_valid = train_test_split(
    datasets["application_train"][selected_features],
    datasets["application_train"]['TARGET'],
    test_size=0.15,
    random_state=42
)

# 4. Also split for the test set if you are using it for evaluation
X_train, X_test, y_train, y_test = train_test_split(
    X_train,
    y_train,
    test_size=0.15,
    random_state=42
)

print(f"Updated X_train shape: {X_train.shape}")
```

```
Updated X_train shape: (222176, 17)
```

Sanity Check

In [33]:
```
# Verify features exist in your training data
if 'RFM_Frequency' in X_train.columns:
    print("✅ Success: X_train has the new features.")
else:
    print("❌ Error: X_train is missing features. Re-run create_rfm_features() and
```

```
✅ Success: X_train has the new features.
```

New Model Run

Logistic 2

In [34]:
```
%%time
# Logistic Regression Model Pipeline

np.random.seed(42)
full_pipeline_with_predictor = Pipeline([
        ("preparation", data_prep_pipeline),
        ("logistic", LogisticRegression(penalty='l2', class_weight='balanced', rand
        )
    ])
model_logistic2 = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
CPU times: total: 3.19 s
Wall time: 3.49 s
```

Random Forrest 2

In [35]:
```
%%time
# Random Forest Model Pipeline

from sklearn.ensemble import RandomForestClassifier
```

```
np.random.seed(42)
full_pipeline_with_predictor = Pipeline([
        ("preparation", data_prep_pipeline),
        ("random_forest", RandomForestClassifier(class_weight='balanced', random_st
    ])
model_rf2 = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
CPU times: total: 39.4 s
Wall time: 43.1 s
```

XGBoost 2

In [36]:
```
%%time
 # XGBoost Model Pipeline
#%pip install xgboost
from xgboost import XGBClassifier


np.random.seed(42)
full_pipeline_with_predictor = Pipeline([
        ("preparation", data_prep_pipeline),
        ("xgb", XGBClassifier(random_state=42,
                              use_label_encoder=False,
                              eval_metric='logloss',scale_pos_weight=11.47))
    ])
model_xgb2 = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
CPU times: total: 6.77 s
Wall time: 1.36 s
```

LightGBM2

In [37]:
```
%%time
# LightGBM Model Pipeline
#%pip install lightgbm
from lightgbm import LGBMClassifier



np.random.seed(42)
full_pipeline_lgbm = Pipeline([# Renamed for ensemble model
        ("preparation", data_prep_pipeline),
        ("lgbm", LGBMClassifier(scale_pos_weight=11.47,random_state=42)) # Scale we
    ])
model_lgbm2 = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
CPU times: total: 7.03 s
Wall time: 1.32 s
```

CatBoost 2

In [38]:
```
%%time
# CatBoost Model Pipeline
#%pip install catboost
```

```
import catboost
np.random.seed(42)
full_pipeline_cat = Pipeline([# Renamed for ensemble model
        ("preparation", data_prep_pipeline),
        ("catboost", catboost.CatBoostClassifier(verbose=0,
        auto_class_weights='Balanced'))
    ])
model_cat2 = full_pipeline_with_predictor.fit(X_train, y_train)
```

```
CPU times: total: 7.25 s
Wall time: 1.44 s
```

New Model Comparison

In [39]:

```
from sklearn.metrics import roc_auc_score, precision_score, recall_score
import pandas as pd
import numpy as np

# ----------------- 1. Define Models and Storage -----------------
# Ensure all these models are trained before running this cell
models = {
    "Logistic Regression": model_logistic,
    "Random Forest": model_rf,
    "XGBoost": model_xgb,
    "LightGBM": model_lgbm,
    "CatBoost": model_cat,
    "Logistic Regression2": model_logistic2,
    "Random Forest2": model_rf2,
    "XGBoost2": model_xgb2,
    "LightGBM2": model_lgbm2,
    "CatBoost2": model_cat2
}

results = []
threshold = 0.5 # Default threshold for classification

# ----------------- 2. Calculate Metrics for Each Model -----------------
# NOTE: Changed X_valid to X_test to match your earlier train_test_split code
print("Starting model evaluation on X_test...")

for name, model in models.items():
    try:
        # A. Get predicted probabilities (required for ROC AUC)
        # The [:, 1] gets the probability of the positive class (Target=1 / Default
        y_pred_proba = model.predict_proba(X_test)[:, 1]

        # B. Get hard predictions (required for Precision and Recall)
        y_pred = (y_pred_proba >= threshold).astype(int)

        # C. Calculate Metrics
        roc_auc = roc_auc_score(y_test, y_pred_proba)
        precision = precision_score(y_test, y_pred, zero_division=0)
        recall = recall_score(y_test, y_pred, zero_division=0)

        # D. Store Results
```

```
        results.append({
            'Model': name,
            'ROC_AUC': roc_auc,
            'Precision (T=0.5)': precision,
            'Recall (T=0.5)': recall
        })

        print(f"-> {name} evaluated.")
    except Exception as e:
        print(f"Error evaluating {name}: {e}")

# ----------------- 3. Display Results -----------------
results_df = pd.DataFrame(results)

# Sort by ROC_AUC to see the best model immediately
results_df_sorted = results_df.sort_values(by='ROC_AUC', ascending=False)

print("\n============================================================")
print("           MODEL PERFORMANCE COMPARISON")
print("============================================================")
display(results_df_sorted)
print("============================================================")
```

```
Starting model evaluation on X_test...
-> Logistic Regression evaluated.
-> Random Forest evaluated.
-> XGBoost evaluated.
-> LightGBM evaluated.
-> CatBoost evaluated.
-> Logistic Regression2 evaluated.
-> Random Forest2 evaluated.
-> XGBoost2 evaluated.
-> LightGBM2 evaluated.
-> CatBoost2 evaluated.

============================================================
           MODEL PERFORMANCE COMPARISON
============================================================
```

	Model	ROC_AUC	Precision (T=0.5)	Recall (T=0.5)
3	LightGBM	0.747997	0.169209	0.680534
4	CatBoost	0.741395	0.175812	0.629517
7	XGBoost2	0.740294	0.174362	0.624962
8	LightGBM2	0.740294	0.174362	0.624962
9	CatBoost2	0.740294	0.174362	0.624962
5	Logistic Regression2	0.738057	0.160582	0.663832
2	XGBoost	0.736451	0.169657	0.623140
0	Logistic Regression	0.736113	0.161223	0.661099
6	Random Forest2	0.726090	0.513514	0.005770
1	Random Forest	0.716173	0.510204	0.007592

==========

> With the New Features added we can seethat LightGBM2 is the new "Best" model. we will be proceeding forward with HyperParameter Tuning on this model

Hyperparameter Tuning

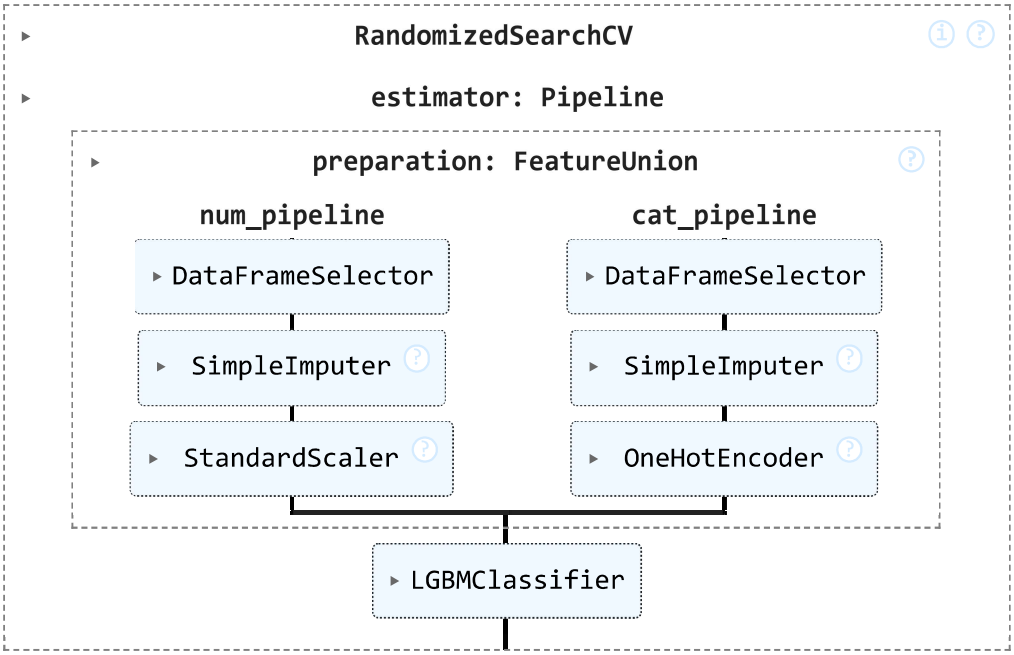
In [40]:
```
from sklearn.model_selection import RandomizedSearchCV
from lightgbm import LGBMClassifier
import time

full_pipeline_lgbm_tune = Pipeline([
    ("preparation", data_prep_pipeline),
    ("lgbm", LGBMClassifier(scale_pos_weight=11.47, random_state=42, verbose=-1, n_
])


param_grid = {
    'lgbm__n_estimators': [200, 500],
    'lgbm__learning_rate': [0.01, 0.05, 0.1],
    'lgbm__num_leaves': [31, 50],
    'lgbm__max_depth': [-1, 10],
    'lgbm__reg_alpha': [0.0, 0.1],
    'lgbm__reg_lambda': [0.0, 0.1]
}

# 3. Setup Randomized Search

random_search = RandomizedSearchCV(
    estimator=full_pipeline_lgbm_tune,
    param_distributions=param_grid,
```

```
    n_iter=5,          # Kept low (5) to ensure it finishes quickly
    cv=3,
    scoring='roc_auc',
    verbose=1,
    n_jobs=-1,
    random_state=42,
    error_score='raise'
)

# 4. Run the Search
print("⏳ Starting Tuning on Full Pipeline...")
start_time = time.time()

# Now fit on X_train (The pipeline will handle the 'object' columns!)
random_search.fit(X_train, y_train)

# 5. Output Results
print(f"✅ Tuning Complete! Time: {round(time.time() - start_time, 2)}s")
print(f"🏆 Best AUC Score: {random_search.best_score_:.4f}")
print("------------------------------------------------")
print(f"⚙️ Best Params: {random_search.best_params_}")
```

```
⏳ Starting Tuning on Full Pipeline...
Fitting 3 folds for each of 5 candidates, totalling 15 fits
✅ Tuning Complete! Time: 33.07s
🏆 Best AUC Score: 0.7454
------------------------------------------------
⚙️ Best Params: {'lgbm__reg_lambda': 0.1, 'lgbm__reg_alpha': 0.0, 'lgbm__num_leave
s': 31, 'lgbm__n_estimators': 200, 'lgbm__max_depth': -1, 'lgbm__learning_rate': 0.0
5}
```

In [41]:
```
random_search
```

Out[41]:



Ensemble Methods

In [42]:
```python
from sklearn.ensemble import VotingClassifier

# ------------------------------------------------------------
# ENSEMBLE: VOTING CLASSIFIER
# ------------------------------------------------------------
# We combine our Top 2 Performers: LightGBM and CatBoost
# This usually gives a boost because they make different "mistakes"

# 1. Define the pipelines (Re-using the ones we defined in the previous step)

estimators_list = [
    ('lgbm', full_pipeline_lgbm),   # Your #1 Model
    ('cat', full_pipeline_cat),     # Your #2 Model

]

# 2. Create the Ensemble
eclf = VotingClassifier(
    estimators=estimators_list,
    voting='soft', # Soft voting averages the probabilities (Best for AUC)
    weights=[1, 1] # Equal weight to both models
)

print("🚂 Training Ensemble (LightGBM + CatBoost)...")
model_ensemble = eclf.fit(X_train, y_train)

# 3. Evaluate
y_pred_proba = model_ensemble.predict_proba(X_test)[:, 1]
ensemble_score = roc_auc_score(y_test, y_pred_proba)

print(f"✅ Ensemble Trained! Combined AUC: {ensemble_score:.4f}")
```

```
🚂 Training Ensemble (LightGBM + CatBoost)...
✅ Ensemble Trained! Combined AUC: 0.7526
```

Feature Importance/Selection

In [43]:
```python
model_lgbm2.named_steps
```

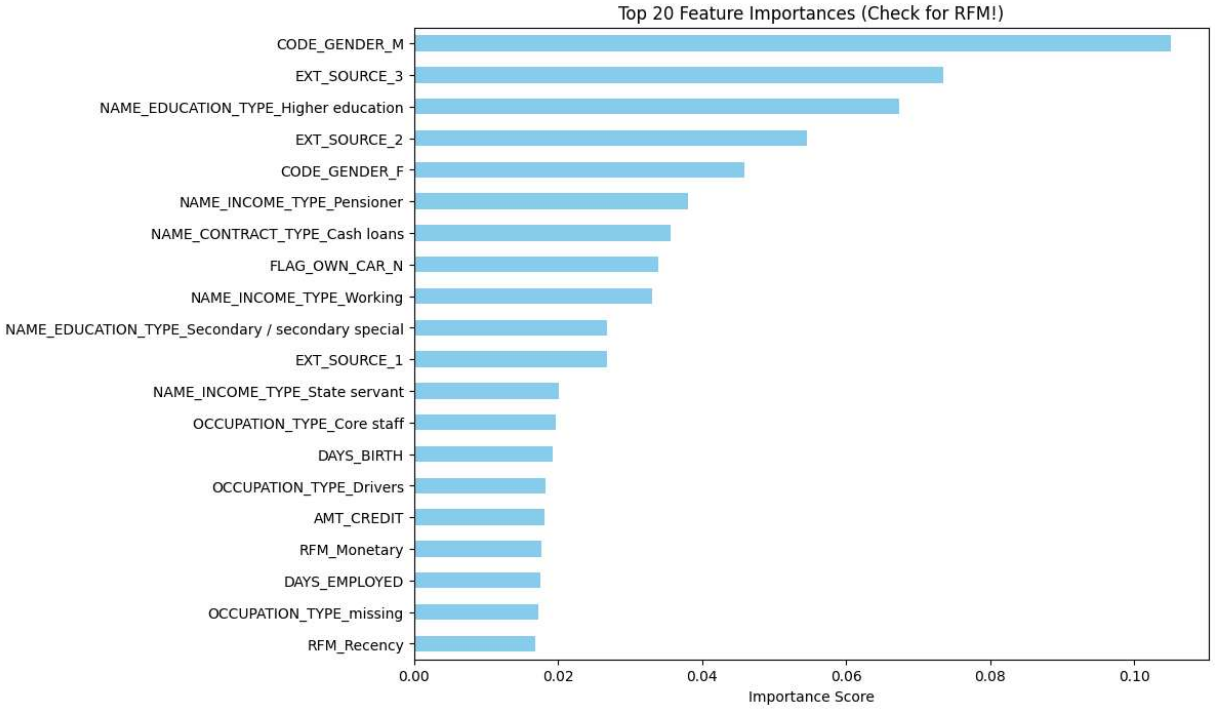
Out[43]:
```
{'preparation': FeatureUnion(transformer_list=[('num_pipeline',
                                        Pipeline(steps=[('selector',
                                                         DataFrameSelector(attribute_name
s=['AMT_INCOME_TOTAL',

'AMT_CREDIT',

'DAYS_EMPLOYED',

'DAYS_BIRTH',

'EXT_SOURCE_1',

'EXT_SOURCE_2',

'EXT_SOURCE_3',

'RFM_Frequency',

'RFM_Monetary',

'RFM_Recency'])),
                                                        ('imputer', SimpleImputer()),
                                                        ('std_scaler',
                                                         StandardScaler())])),
                                       ('cat_pipeline',
                                        Pipeline(steps=[('selector',
                                                         DataFrameSelector(attribute_name
s=['CODE_GENDER',

'FLAG_OWN_REALTY',

'FLAG_OWN_CAR',

'NAME_CONTRACT_TYPE',

'NAME_EDUCATION_TYPE',

'OCCUPATION_TYPE',

'NAME_INCOME_TYPE'])),
                                                        ('imputer',
                                                         SimpleImputer(fill_value='missin
g',
                                                                       strategy='constan
t')),
                                                        ('ohe',
                                                         OneHotEncoder(handle_unknown='ig
nore',
                                                                       sparse_output=Fals
e))]))]),
 'xgb': XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, gamma=None, grow_policy=None,
```

```
                importance_type=None, interaction_constraints=None,
                learning_rate=None, max_bin=None, max_cat_threshold=None,
                max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
                max_leaves=None, min_child_weight=None, missing=nan,
                monotone_constraints=None, multi_strategy=None, n_estimators=None,
                n_jobs=None, num_parallel_tree=None, random_state=42, ...)}
```

In [44]:

```python
import matplotlib.pyplot as plt
import pandas as pd

# ------------------------------------------------------------
# FEATURE IMPORTANCE VISUALIZATION
# ------------------------------------------------------------

# 1. Access the trained model and transformers from the Pipeline
# We use LightGBM because it was your best single model
model = model_lgbm2.named_steps['xgb']
importances = model.feature_importances_

# 2. Reconstruct the Feature Names
# (Since OHE created new columns, we need to ask the pipeline what they are named)
prep_step = model_lgbm2.named_steps['preparation']

# Access the OneHotEncoder from the categorical branch
cat_pipe = dict(prep_step.transformer_list)['cat_pipeline']
ohe_step = cat_pipe.named_steps['ohe']
cat_feature_names = ohe_step.get_feature_names_out(cat_attribs)

# Combine: Numeric Features (kept as is) + Categorical Features (expanded)
# Note: 'num_attribs' must match the list you defined in your pipeline earlier
all_feature_names = num_attribs + list(cat_feature_names)

# 3. Create the Importance Series
feat_imp = pd.Series(importances, index=all_feature_names).sort_values(ascending=Fa

# 4. Print Specific Rank of RFM Features (For your Report)
print("--- RFM Feature Ranks ---")
for rfm in ['RFM_Frequency', 'RFM_Monetary', 'RFM_Recency']:
    try:
        rank = list(feat_imp.index).index(rfm) + 1
        score = feat_imp[rfm]
        print(f"Feature: {rfm} | Rank: #{rank} | Importance: {score}")
    except:
        print(f"Feature {rfm} not found in model.")

# 5. Plot Top 20
plt.figure(figsize=(10, 8))
feat_imp.head(20).plot(kind='barh', color='skyblue')
plt.title('Top 20 Feature Importances (Check for RFM!)')
plt.xlabel('Importance Score')
plt.gca().invert_yaxis() # Puts the #1 feature at the top
plt.show()
```

```
--- RFM Feature Ranks ---
Feature: RFM_Frequency | Rank: #28 | Importance: 0.013742746785283089
Feature: RFM_Monetary | Rank: #17 | Importance: 0.017717435956001282
Feature: RFM_Recency | Rank: #20 | Importance: 0.016871420666575432
```



Master Model Compare

In [45]:

```python
from sklearn.metrics import roc_auc_score, precision_score, recall_score
import pandas as pd
import numpy as np

# ----------------- 1. Define Models and Storage -----------------
# Ensure all these models are trained before running this cell
models = {
    "Logistic Regression": model_logistic,
    "Random Forest": model_rf,
    "XGBoost": model_xgb,
    "LightGBM": model_lgbm,
    "CatBoost": model_cat,
    "Logistic Regression2": model_logistic2,
    "Random Forest2": model_rf2,
    "XGBoost2": model_xgb2,
    "LightGBM2": model_lgbm2,
    "CatBoost2": model_cat2,
    "Ensemble (LGBM + Cat)": model_ensemble  # The new addition
}

results = []
threshold = 0.5 # Default threshold for classification

# ----------------- 2. Calculate Metrics for Each Model -----------------
# NOTE: Changed X_valid to X_test to match your earlier train_test_split code
print("Starting model evaluation on X_test...")
```

```python
for name, model in models.items():
    try:
        # A. Get predicted probabilities (required for ROC AUC)
        # The [:, 1] gets the probability of the positive class (Target=1 / Default
        y_pred_proba = model.predict_proba(X_test)[:, 1]

        # B. Get hard predictions (required for Precision and Recall)
        y_pred = (y_pred_proba >= threshold).astype(int)

        # C. Calculate Metrics
        roc_auc = roc_auc_score(y_test, y_pred_proba)
        precision = precision_score(y_test, y_pred, zero_division=0)
        recall = recall_score(y_test, y_pred, zero_division=0)

        # D. Store Results
        results.append({
            'Model': name,
            'ROC_AUC': roc_auc,
            'Precision (T=0.5)': precision,
            'Recall (T=0.5)': recall
        })

        print(f"-> {name} evaluated.")
    except Exception as e:
        print(f"Error evaluating {name}: {e}")

# ----------------- 3. Display Results -----------------
results_df = pd.DataFrame(results)

# Sort by ROC_AUC to see the best model immediately
results_df_sorted = results_df.sort_values(by='ROC_AUC', ascending=False)

print("\n============================================================")
print("           MODEL PERFORMANCE COMPARISON")
print("============================================================")
display(results_df_sorted)
print("============================================================")
```

```
Starting model evaluation on X_test...
-> Logistic Regression evaluated.
-> Random Forest evaluated.
-> XGBoost evaluated.
-> LightGBM evaluated.
-> CatBoost evaluated.
-> Logistic Regression2 evaluated.
-> Random Forest2 evaluated.
-> XGBoost2 evaluated.
-> LightGBM2 evaluated.
-> CatBoost2 evaluated.
-> Ensemble (LGBM + Cat) evaluated.

============================================================
           MODEL PERFORMANCE COMPARISON
============================================================
```

	Model	ROC_AUC	Precision (T=0.5)	Recall (T=0.5)
10	Ensemble (LGBM + Cat)	0.752595	0.178315	0.656241
3	LightGBM	0.747997	0.169209	0.680534
4	CatBoost	0.741395	0.175812	0.629517
7	XGBoost2	0.740294	0.174362	0.624962
8	LightGBM2	0.740294	0.174362	0.624962
9	CatBoost2	0.740294	0.174362	0.624962
5	Logistic Regression2	0.738057	0.160582	0.663832
2	XGBoost	0.736451	0.169657	0.623140
0	Logistic Regression	0.736113	0.161223	0.661099
6	Random Forest2	0.726090	0.513514	0.005770
1	Random Forest	0.716173	0.510204	0.007592

==========

> Note the Ensemble and 2 models have added features, so the Ensemble Model is technically the best model

Phase 4

Step 1

In [46]:
```
#pip uninstall torch torchvision torchaudio
```

In [47]:
```
#%pip install torch torchvision torchaudio --index-url https://download.pytorch.org
```

Import

In [48]:
```
#%pip install torch torchvision torchaudio
```

Sanity check

In [49]:
```
import torch
print(torch.__version__)
```

```
2.6.0+cpu
```

Step 1

In [50]:
```
import torch
from torch.utils.data import DataLoader, TensorDataset
import numpy as np

# 1. Transform the data with the existing Phase 3 pipeline
# Used Pipeline from Cell ~33 of the notebook
print("Transforming data via Phase 3 pipeline...")
X_train_processed = data_prep_pipeline.fit_transform(X_train)
X_test_processed = data_prep_pipeline.transform(X_test)
#X_kaggle_processed = data_prep_pipeline.transform(X_kaggle_test)

# 2. Convert to PyTorch Tensors
# Note: PyTorch expects Float32 by default for neural networks
train_features = torch.tensor(X_train_processed, dtype=torch.float32)
train_targets = torch.tensor(y_train.values, dtype=torch.float32).unsqueeze(1) # Sh

test_features = torch.tensor(X_test_processed, dtype=torch.float32)
test_targets = torch.tensor(y_test.values, dtype=torch.float32).unsqueeze(1)

# 3. Create DataLoaders
# Batch size is a hyperparameter
BATCH_SIZE = 1024

train_dataset = TensorDataset(train_features, train_targets)
test_dataset = TensorDataset(test_features, test_targets)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_w
val_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_wor

print(f"Input Feature Dimension: {train_features.shape[1]}")
```

```
Transforming data via Phase 3 pipeline...
Input Feature Dimension: 51
```

Step 2

In [51]:
```
#%pip install pytorch-lightning torchmetrics
import pytorch_lightning as pl
import torch.nn as nn
import torch.nn.functional as F
from torch.optim import Adam
from torchmetrics.classification import BinaryAUROC
```

In [52]:
```
class HCDRModel(pl.LightningModule):
    def __init__(self, input_dim, learning_rate=1e-3):
        super().__init__()
        self.save_hyperparameters()

        # --- Architecture Definition ---
```

```
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.BatchNorm1d(256), # Helps with convergence
            nn.ReLU(),
            nn.Dropout(0.3),     # Regularization

            nn.Linear(256, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(128, 64),
            nn.ReLU(),

            nn.Linear(64, 1)     # Output: 1 raw logit
        )

        # Weighted Loss for Class Imbalance

        self.criterion = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([11.0]))

        # Metric
        self.auc = BinaryAUROC()

    def forward(self, x):
        return self.net(x)

    def training_step(self, batch, batch_idx):
        x, y = batch
        logits = self(x)
        loss = self.criterion(logits, y)

        # Log to TensorBoard
        self.log("train_loss", loss, prog_bar=True)
        return loss

    def validation_step(self, batch, batch_idx):
        x, y = batch
        logits = self(x)
        loss = self.criterion(logits, y)
        preds = torch.sigmoid(logits)

        # Calculate AUC
        self.auc(preds, y)

        # Log metrics
        self.log("val_loss", loss, prog_bar=True)
        self.log("val_auc", self.auc, on_step=False, on_epoch=True, prog_bar=True)
        return loss

    def configure_optimizers(self):
        return Adam(self.parameters(), lr=self.hparams.learning_rate)
```

Step3

In [53]:

```python
from pytorch_lightning.loggers import TensorBoardLogger
from pytorch_lightning.callbacks import ModelCheckpoint, EarlyStopping

# 1. Setup Logger
logger = TensorBoardLogger("tb_logs", name="hcdr_model")

# 2. Callbacks (Stop if validation stops improving)
checkpoint_callback = ModelCheckpoint(
    monitor="val_auc",
    mode="max",
    filename="best-hcdr-{epoch:02d}-{val_auc:.4f}"
)

early_stop_callback = EarlyStopping(
    monitor="val_auc",
    patience=5,
    mode="max"
)

# 3. Initialize Trainer
trainer = pl.Trainer(
    max_epochs=20,
    accelerator="auto", # Uses GPU if available-Note Available
    logger=logger,
    callbacks=[checkpoint_callback, early_stop_callback],
    log_every_n_steps=10
)

# 4. Initialize Model
input_dim = train_features.shape[1]
model = HCDRModel(input_dim=input_dim)

# 5. Train
trainer.fit(model, train_loader, val_loader)

print(f"Best Validation AUC: {checkpoint_callback.best_model_score}")
```

```
GPU available: False, used: False
TPU available: False, using: 0 TPU cores
```

	Name	Type	Params	Mode	FLOPs
0	net	Sequential	55.3 K	train	0
1	criterion	BCEWithLogitsLoss	0	train	0
2	auc	BinaryAUROC	0	train	0

```
Trainable params: 55.3 K
Non-trainable params: 0
Total params: 55.3 K
Total estimated model params size (MB): 0
Modules in train mode: 14
Modules in eval mode: 0
Total FLOPs: 0
```

Output()

```
Best Validation AUC: 0.744864284992218
```

In [54]:

```python
import pandas as pd
import torch

# 1. Evaluation mode
model.eval()

# 2. Process Kaggle test data
# First, add RFM features to the test data
print("Adding RFM features to Kaggle test data...")
datasets['application_test'] = create_rfm_features(datasets['bureau'], datasets['ap
X_kaggle_test_with_rfm = datasets["application_test"][selected_features]

# Note: Ensure 'data_prep_pipeline' was fitted on X_train previously
print("Processing Kaggle test data...")
X_kaggle_processed = data_prep_pipeline.transform(X_kaggle_test_with_rfm)
kaggle_features = torch.tensor(X_kaggle_processed, dtype=torch.float32)

# 3. Predictions
print("Generating predictions...")
with torch.no_grad():
    # Pass data through model
    logits = model(kaggle_features)
    # Apply Sigmoid
    kaggle_preds = torch.sigmoid(logits).numpy().flatten()

# 4. Create Submission DataFrame
submission = pd.DataFrame({
    'SK_ID_CURR': datasets['application_test']['SK_ID_CURR'],
    'TARGET': kaggle_preds
})

# 5. Save to CSV
submission_filename = "submission_phase4_deeplearning.csv"
submission.to_csv(submission_filename, index=False)

print(f"✅ Submission file saved: {submission_filename}")
print(f"Shape: {submission.shape}")
print(submission.head())
```

```
Adding RFM features to Kaggle test data...
Processing Kaggle test data...
Generating predictions...
✅ Submission file saved: submission_phase4_deeplearning.csv
Shape: (48744, 2)
   SK_ID_CURR    TARGET
0      100001  0.372519
1      100005  0.772501
2      100013  0.272279
3      100028  0.323236
4      100038  0.659471
```